# MuSEUM 実験
# 計測・制御プログラム操作マニュアル

第 2.2 版

最終更新：2026 年 11 月 6 日

## 更新履歴

| 版番号 | 更新日 | 変更内容 |
|---|---|---|
| 2.0 | 2026 年 10 月 26 日 | LaTeX 版を作成。 |
| 2.1 | 2026 年 10 月 26 日 | 表題と更新履歴の記載を修正。 |
| 2.2 | 2026 年 11 月 6 日 | Linux 直接接続・tools による個別操作に変更。最大反復 10 回、Tuning 0 開始・完了 PNG の表示、パワー制御の引数省略を反映。 |

# 目次


1 スイッチの切り替え：Switchcontrol.py 4

1.1 特性評価/実験、TM110/TM210 を切り替える . . . . . 4

1.2 スイッチ用電源の CH2・CH3・CH4 の出力を全て OFF にする . . . . . 4

1.3 python 起動後 . . . . . 4

2 VNA の設定・測定・CSV 保存：test_main.py 5

2.1 VNA 接続 → 測定条件の設定 → 全 4ch を各 10 回測定 → CSV 保存をまとめて行う . 5

3 VNA の個別操作：vna_tools.py 5

3.1 Python 起動後、VNA と接続 . . . . . 5

3.2 測定条件・トレース・画面・マーカーを設定する . . . . . 5

3.3 全 4ch を各 10 回測定し、CSV を保存する . . . . . 6

3.4 現在の Narrow S11 から、最小振幅の周波数を取得する . . . . . 6

3.5 操作終了後、通信を閉じる . . . . . 6

4 Piezo 調整 6

4.1 スイッチを VNA 側・使用するモードに切り替える . . . . . 6

4.2 ANC350 を Linux PC に接続する . . . . . 7

4.3 Linux PC で受信プログラムを起動する：frequency_controller.py . . . . . 7

4.4 別タブで Python を起動し、送信側を準備する . . . . . 7

4.5 python 起動後 . . . . . 7

4.6 送信側の通信を閉じる . . . . . 7

4.7 調整条件と保存ファイル . . . . . 8

5 Piezo の個別操作 8

5.1 スイッチを VNA 側・使用するモードに切り替える . . . . . 8

5.2 ANC350 を Linux PC に接続する . . . . . 8

5.3 Linux PC で Python を起動し、ANC350 に接続する . . . . . 8

5.4 現在位置・駆動設定を確認する . . . . . 9

5.5 ステップ電圧・周波数を設定する . . . . . 9

5.6 指定した step 数だけ移動する . . . . . 9

5.7 初期位置へ戻す . . . . . 9
5.8 位置・駆動設定を保存する . . . . . 10
5.9 操作終了後、通信を閉じる . . . . . 10

6 パワーの feedback 10
6.1 パワー測定値の送信を開始する：RaS_NRP2_Operator.py . . . . . 10
6.2 スイッチを AMP 側・使用するモードに切り替える . . . . . 10
6.3 別タブで Python を起動し、フィードバックを準備する . . . . . 11
6.4 python 起動後 . . . . . 11
6.5 設定の変更 . . . . . 11
6.6 現在の状態を確認する . . . . . 12
6.7 フィードバックを停止し、SG 出力を OFF にする . . . . . 12
6.8 ログのパスを確認 . . . . . 12
6.9 測定ログのリアルタイム確認 . . . . . 13

7 1port 運用 13
7.1 スイッチを切り替える . . . . . 13
7.2 Piezo 調整の受信側を起動する . . . . . 13

# 1　スイッチの切り替え：Switchcontrol.py

## 1.1　特性評価/実験、TM110/TM210 を切り替える

```
% python Switchcontrol.py AMP TM110
```

- 引数 1 VNA or AMP
- 引数 2 TM110 or TM210

## 1.2　スイッチ用電源の CH2・CH3・CH4 の出力を全て OFF にする

```
% python Switchcontrol.py OFF
```

## 1.3　python 起動後

```
>>>import Switchcontrol

>>>Switchcontrol.set_switch("VNA", "TM110")
```

- 引数 1 VNA or AMP
- 引数 2 TM110 or TM210

または

```
>>>Switchcontrol.all_off()
```

# 2　VNA の設定・測定・CSV 保存：test_main.py

## 2.1　VNA 接続 → 測定条件の設定 → 全 4ch を各 10 回測定 → CSV 保存をまとめて行う

```
% python test_main.py
```

- 保存ファイル名の末尾を指定する

```
% python test_main.py TM110_check
```

引数：ファイル名の末尾に付ける文字列

# 3　VNA の個別操作：vna_tools.py

## 3.1　Python 起動後、VNA と接続

```
>>> import pyvisa
>>> import vna_tools
>>> rm = pyvisa.ResourceManager('@py')
>>> znb = vna_tools.get_vna_resource(rm, 'TCPIP0::192.168.12.4::inst0::INSTR')
```

## 3.2　測定条件・トレース・画面・マーカーを設定する

```
>>> vna_tools.setup_vna(znb)
```

- TM110/TM210 それぞれの Wide/Narrow を全 4ch に設定する。

### 3.3 全 4ch を各 10 回測定し、CSV を保存する

```
>>> csv_path = vna_tools.measure_and_save(znb)
```

### 3.4 現在の Narrow S11 から、最小振幅の周波数を取得する

```
>>> f0, amp_min = vna_tools.find_min_freq(znb, 2, "Trc4")
```

- TM110：2, ”Trc4”
- TM210：4, ”Trc9”

### 3.5 操作終了後、通信を閉じる

```
>>> znb.close()

>>> rm.close()
```

## 4 Piezo 調整

### 4.1 スイッチを VNA 側・使用するモードに切り替える

```
% python Switchcontrol.py VNA TM110
```

- TM210 の場合は TM210 を指定する。

## 4.2 ANC350 を Linux PC に接続する

ANC350 を USB で Linux PC に直接接続する。Windows PC での事前接続・初期化と、USB の差し替えは不要。

## 4.3 Linux PC で受信プログラムを起動する：frequency_controller.py

```
% python frequency_controller.py
```

- Waiting for frequency command... と表示されたら、別タブから指示を送る。
- このターミナルは起動したままにする。
- この受信側のターミナルに、現在の周波数・目標との差・移動 step 数・最終結果が表示される。
- 調整終了後は、次の指示を待ち受ける。

## 4.4 別タブで Python を起動し、送信側を準備する

## 4.5 python 起動後

```
>>> from frequency_command import FrequencyCommandProducer

>>> producer = FrequencyCommandProducer()

>>> producer.send_target("TM110", 1897400)
```

- 引数 1：TM110 or TM210
- 引数 2：目標周波数［kHz］

## 4.6 送信側の通信を閉じる

```
>>> producer.close()
```

### 4.7 調整条件と保存ファイル

- 目標との差が ±10 kHz 以内で完了。最大反復回数は 10 回。
- 正負の slope 測定後、フィードバック移動前のグラフを `tuning00` として保存する。
- 以降は `tuning01`、`tuning02` と続く。
- 完了した PNG は `tuning03_final.png` などとなり、図にも (Final) を表示する。

各回の移動前に VNA を 10 回測定し、平均 CSV・PNG を controller と同じ場所の `data/YYYYMMDD/`、生データ CSV をその中の `raw_data/` に保存する。CSV には final を付けない。ピエゾの位置・駆動設定は自動保存しない。controller は駆動電圧・周波数を自動設定しないため、変更する場合は起動前に設定する。

## 5 Piezo の個別操作

### 5.1 スイッチを VNA 側・使用するモードに切り替える

```
% python Switchcontrol.py VNA TM110
```

- TM210 の場合は TM210 を指定する。

### 5.2 ANC350 を Linux PC に接続する

ANC350 を USB で Linux PC に直接接続する。Windows PC での事前接続・初期化と、USB の差し替えは不要。

### 5.3 Linux PC で Python を起動し、ANC350 に接続する

```
>>> import piezo_tools

>>> atc = piezo_tools.connect_piezo()
```

- frequency_controller.py との同時接続はできない

TM110：axis1、TM210：axis2

## 5.4 現在位置・駆動設定を確認する

```
>>> piezo_tools.get_position(atc)
>>> piezo_tools.get_drive_settings(atc)
```

全軸の位置［m］、ステップ電圧・周波数などを表示する。

## 5.5 ステップ電圧・周波数を設定する

```
>>> piezo_tools.set_step_voltage(atc, 1, 30.0)
>>> piezo_tools.set_step_frequency(atc, 1, 1500)
```

第 2 引数は軸番号。電圧［V］と周波数［Hz］を設定する。設定変更後は slope を測り直す（自動調整の開始時に測定される）。

## 5.6 指定した step 数だけ移動する

```
>>> piezo_tools.move_steps(atc, axis, 10, delay=0.01, settle=1.0)
```

- atc の次の引数：軸番号
- その次の引数：移動 step 数。負方向なら-10 などを指定する。
- delay：step 間の待ち時間、settle：移動後の待ち時間［秒］

## 5.7 初期位置へ戻す

```
>>> ok = piezo_tools.reset_position(atc, axes=(1,))
```

axes=(1,) は TM110 のみ。axes を省略すると TM110・TM210 の両軸を戻す。初期位置は TM110 が 7.090 mm、TM210 が 4.185 mm。許容誤差は 50 $\mu$m、待機上限は 30 秒。現在位置から 0.5 mm を超えて離れた目標への移動は拒否される。

### 5.8 位置・駆動設定を保存する

```
>>> means = piezo_tools.measure_and_save(atc, suffix_arg="check")
```

実行フォルダの data/YYYYMMDD/RunNNN/ に平均位置・駆動設定と生データの CSV を保存する。

### 5.9 操作終了後、通信を閉じる

```
>>> piezo_tools.disconnect_piezo(atc)
```

## 6 パワーの feedback

### 6.1 パワー測定値の送信を開始する：RaS_NRP2_Operator.py

```
% python RaS_NRP2_Operator.py
```

- このターミナルは起動したままにする。

### 6.2 スイッチを AMP 側・使用するモードに切り替える

```
% python Switchcontrol.py AMP TM110
```

- TM210 の場合は TM210 を指定する。

## 6.3 別タブで Python を起動し、フィードバックを準備する

## 6.4 python 起動後

```
>>> from power_feedback import PowerFeedbackController

>>> feedback = PowerFeedbackController()

>>> feedback.start(frequency_ghz=1.8974, target_power_w=7e-6,
↪  initial_amp_power_dbm=-25.0)
```

- frequency_ghz：AMP の周波数［GHz］。必須。省略できない。
- target_power_w：NRP2 CH2 で測定する目標パワー［W］。省略時は現在の設定値を使う（初期値 7 $\mu$W）。
- initial_amp_power_dbm：開始時の AMP 出力［dBm］。省略時は既定値 −30 dBm。ただし現在の下限制限により実際は −25 dBm。
- 周波数設定 → 初期 AMP 出力設定 → AMP 出力 ON → フィードバック開始まで行う。

デフォルト値と AMP 出力の上限・下限は、power_feedback.py の冒頭にまとめて記載している。

周波数だけを指定して開始する場合：

```
>>> feedback.start(frequency_ghz=1.8974)
```

## 6.5 設定の変更

変更後の設定は、次の制御計算から使われる

目標パワーを変更する

```
>>> feedback.set_target_power(7e-6)
```

- 引数：目標パワー［W］

PID の係数を変更する

```
>>> feedback.set_pid(kp=1.0, ki=0.0, kd=0.0)
```

制御の更新間隔を変更する

```
>>> feedback.set_interval(1.0)
```

- 引数：更新ごとの待ち時間［秒］

## 6.6 現在の状態を確認する

```
>>> feedback.get_status()
```

- AMP 出力 ON/OFF、フィードバック状態、周波数、AMP 出力、目標値、最新の測定値などを表示する。

## 6.7 フィードバックを停止し、SG 出力を OFF にする

```
>>> feedback.stop()
```

## 6.8 ログのパスを確認

```
>>> print(feedback.log_path)
```

### 6.9　測定ログのリアルタイム確認

別のタブで

```
% tail -f ログのパス
```

## 7　1port 運用

TM110・TM210 の両方を、TM110 側の入力ポートで測定する。

### 7.1　スイッチを切り替える

```
% python Switchcontrol_1port.py VNA TM110
```

TM110 ／ TM210 のどちらを指定しても、物理入力は TM110 側に固定される。

### 7.2　Piezo 調整の受信側を起動する

```
% python frequency_controller_1port.py
```

送信方法・周波数の単位・調整条件は通常版と同じ。保存する CSV・PNG の名前に 1port_input_TM110 が付き、図にも入力ポートを表示する。